

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

"ЛР 3: Контейнеризация написанного приложения
средствами docker"

Выполнил:

Михайлов Юрий

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Ход работы

На основе микросервисной архитектуры, разработанной в ДЗ4 и реализованной в ЛР2 и ЛР5, выполнена полная контейнеризация всех компонентов приложения с использованием Docker.

1) Реализация Dockerfile для каждого сервиса:

Для каждого из четырех сервисов (User Service, Property Service, Booking Service, Engagement Service) создан Dockerfile на основе образа node:20-alpine. В каждом Dockerfile выполнены следующие шаги: установка рабочей директории /app, копирование package.json и package-lock.json, установка зависимостей через npm ci, копирование исходного кода, сборка TypeScript через npm run build, открытие соответствующего порта (8000-8003) и запуск сервиса через npm start. Выбор alpine-версии Node.js обусловлен минимальным размером образа и достаточностью для лабораторной работы.

2) Написание общего docker-compose.yml:

Создан единый docker-compose.yml, который описывает все компоненты системы:

- RabbitMQ - брокер сообщений с management-плагином, порты 5672 (AMQP) и 15672 (UI), добавлен healthcheck для контроля готовности.
- Четыре базы данных PostgreSQL (auth-db, property-db, booking-db, engagement-db) на образе postgres:16-alpine. Каждой базе назначены внешние порты 15432-15435 соответственно и выделены именованные тома для сохранения данных.
- Четыре сервиса (auth-service, property-service, booking-service, engagement-service), каждый собирается из соответствующей

директории, пробрасывает свой порт (8000-8003) и зависит от своей базы данных и RabbitMQ с условием `service_healthy`.

3) Настройка сетевого взаимодействия между сервисами:

- Для обеспечения связи между контейнерами создана отдельная сеть `rental-network` с драйвером `bridge`. Все сервисы и базы данных подключены к этой сети. Внутри сети контейнеры обращаются друг к другу по именам сервисов (например, `auth-service`, `property-db`, `rabbitmq`) вместо IP-адресов.
- Для корректной работы внутри Docker-контейнеров внесены следующие изменения в конфигурацию:
- В `.env` файлах всех сервисов параметр `DATABASE_HOST` заменен с `localhost` на имя соответствующего контейнера БД (`auth-db`, `property-db` и т.д.), а `DATABASE_PORT` установлен в 5432 (внутренний порт PostgreSQL).
- В `.env` файлах всех сервисов параметр `RABBITMQ_URL` заменен на `amqp://guest:guest@rabbitmq:5672`.
- В `.env` файлах всех сервисов параметры URL зависимых сервисов (`AUTH_SERVICE_URL`, `PROPERTY_SERVICE_URL`, `BOOKING_SERVICE_URL`, `ENGAGEMENT_SERVICE_URL`) заменены с `localhost` на имена соответствующих контейнеров.
- В `settings.ts` всех сервисов параметр `APP_HOST` изменен с `127.0.0.1` на `0.0.0.0`, чтобы сервис слушал все сетевые интерфейсы внутри контейнера.
- Добавлена переменная `NODE_ENV=production` в `.env` файлы для использования скомпилированных `.js` файлов вместо `.ts`.

Вывод

В результате выполнения работы все четыре микросервиса успешно контейнеризированы, работают в изолированных средах, имеют собственные базы данных, общаются между собой через единую Docker-сеть и используют RabbitMQ для асинхронных событий. Система

запускается одной командой `docker-compose up`. В ходе работы были усовершенствованы навыки работы с Docker, окружением и его развертыванием.